

Bachelorarbeit

Ansatz zur Optimierung einer CI-Pipeline mit reproduzierbaren lokalen Builds

Ruben Leander Rosenmeyer

Gutachter: Prof. Dr. Peter Thiemann

Betreuer: Dr. Michael Sperber

Albert-Ludwigs-Universität Freiburg

Technische Fakultät

Institut für Informatik

13. Juli 2026

Bearbeitungszeit

13. 04. 2026 – 13. 07. 2026

Gutachter

Prof. Dr. Peter Thiemann

Betreuer

Dr. Michael Sperber

ERKLÄRUNG

Hiermit erkläre ich, dass ich diese Abschlussarbeit selbständig verfasst habe, keine anderen als die angegebenen Quellen/Hilfsmittel verwendet habe und alle Stellen, die wörtlich oder sinngemäß aus veröffentlichten Schriften entnommen wurden, als solche kenntlich gemacht habe.

Darüber hinaus erkläre ich, dass diese Abschlussarbeit nicht, auch nicht auszugsweise, bereits für eine andere Prüfung angefertigt wurde.

Ort, Datum

Unterschrift

Zusammenfassung

Diese Bachelorarbeit beschäftigt sich mit der Erstellung eines Tools, welches anhand der Ausgabe eines vorherigen Testlaufes entscheiden kann, welche Testfälle innerhalb eines Projektes bereits abgelaufen sind, und welche nicht. Dabei soll das Tool unabhängig vom dem oder den verwendeten Testframework(s) sein, und sich in eine bestehende CI-Pipeline integrieren lassen, auch hier unabhängig von der verwendeten Software.

Dadurch soll die Gesamtzahl der Testfälle, die abgelaufen werden müssen, reduzieren lassen, wodurch sich auch die Laufzeit der Testsphase bzw. der gesamten Pipeline verkürzt.

Hierfür werden die Anforderungen an ein Projekt, in das das Tool integriert werden kann, beschrieben und erklärt, sowie unterschiedliche Testausgabeformate hinsichtlich ihrer Eignung miteinander verglichen und diskutiert.

Inhaltsverzeichnis

Zusammenfassung	ii
1 Einleitung	1
1.1 Problemstellung	1
1.2 Zielsetzung	1
2 Hintergrund	3
2.1 Anforderungen an ein Testausgabeformat	3
2.2 Vergleich von Testausgabeformaten	3
2.2.1 JUnit XML	3
2.2.2 Test Anything Protocol (TAP)	4
2.2.3 JSON basierte Formate	4
2.3 Entscheidung	4
2.4 JUnit XML im Detail	4
2.5 git notes	4
3 Umsetzung	5
3.1 Anforderungen an ein Projekt	5
3.1.1 Deterministische builds - 'it works on my machine' . .	5
3.1.2 test discovery	5
3.1.3 test execution	6
3.2 Anforderungen an ein Framework	6

3.3	Einschränkungen	6
3.4	Codestruktur	6
4	Related Work	7
4.1	Andere Tools	7
5	Fazit	8
5.1	Zusammenfassung der Ergebnisse	8
5.2	Möglichkeiten zur Verbesserung	8
	Bibliography	8

1 Einleitung

1.1 Problemstellung

Jedes größere Softwareprojekt benötigt eine Vielzahl von Tests, um die Qualität der Software zu sichern. Dabei werden die einzelnen Tests für gewöhnlich in verschiedene Klassen unterteilt, die verschiedene Abschnitte der Software abdecken. Die meisten gängigen Testframeworks bieten die Möglichkeit, einzelne Testklassen und/oder Testfälle gezielt auszuführen, ohne die restlichen Testfälle dabei mit auszuführen.

Doch diese Praxis birgt zum einen die Gefahr, dass manche Tests vom Entwickler vergessen werden können. Andererseits ist es manchmal auch gewünscht, dass nur gewisse Tests ausgeführt werden, beispielsweise bei einer langen Laufzeit der Tests. Dies kann dazu führen, dass die Tests nicht in ihrer Gesamtheit ausgeführt werden, und Fehler unentdeckt im Code verbleiben können.

1.2 Zielsetzung

Das Ziel des im Rahmen dieser Bachelorarbeit entwickelten Tools ist, eine verlässliche Aussage darüber treffen zu können, welche Tests in der aktuellen Iteration bereits ausgeführt wurden, und genau anzugeben, welche Testfälle zum Erreichen einer 100%igen Testabdeckung noch ausgeführt werden müssen.

Dabei soll die Wahl des Teastframeworks möglichst keine Rolle spielen.
Außerdem soll das Tool in eine bestehende CI-Pipeline integriert werden können.

2 Hintergrund

2.1 Anforderungen an ein Testausgabeformat

Mit Blick auf die Anforderungen an das Tool, frameworkübergreifend ein korrektes Ergebnis zu liefern, ergeben sich folgende Anforderungen an ein potentielle nutzbare Ausgabeformat:

Ein ideales Ausgabeformat für Tests:

- kennt die Gesamtmenge aller Tests im Projekt
- sagt für jeden Test einzelnd aus, ob er ausgeführt wurde oder nicht
- ist in vielen Frameworks über viele Sprachen hinweg vertreten

2.2 Vergleich von Testausgabeformaten

kommt rein:

-) kurze Historie/Vorstellung
-) vorgesehene Verwendung/Verbreitung
-) Aufbau/Struktur

2.2.1 JUnit XML

SUnit -> xUnit, JUnit & NUnit etc.

2.2.2 Test Anything Protocol (TAP)

2.2.3 JSON basierte Formate

-> kleine Tabelle zum Vergleich der 3 als Übersicht

2.3 Entscheidung

- erfüllt nicht alle Anforderungen, aber erklären, warum gut genug

2.4 JUnit XML im Detail

- Genaue Struktur (unterschiedliche root tags!)

- Auf unterschiedliche Umsetzungen eingehen

2.5 git notes

3 Umsetzung

3.1 Anforderungen an ein Projekt

3.1.1 Deterministische builds - 'it works on my machine'

Damit das Tool eine qualifizierte Aussage über die Testergebnisse treffen kann, muss gewährleistet sein, dass alle Tests unabhängig von der Entwicklungsumgebung immer das selbe Testergebnis liefern. Ein Testfall, der auf der Machine eines Entwicklers ein bestimmtes Ergebnis liefert, muss auch zwingend auf einem Build-Server das selbe Ergebnis liefern. Andernfalls könnte eine Diskrepanz zwischen den Testergebnissen zu einer verfälschten Aussage des Tools über den aktuellen Stand der Test führen.

Da Junit XML nicht alle in 2.1 aufgeführten Anforderungen erfüllt, ergeben sich einige weitere Anforderungen an ein Projekt, das das Tool nutzen möchte, um trotzdem das geforderte Ziel zu erreichen.

3.1.2 test discovery

Damit das Tool die Gesamtmenge aller Tests im Projekt kennt, muss im Projekt ein Script vorhanden sein, welches die die Gesamtmenge aller Tests in einem spezifiziertem Format angibt.

-> qualifiedname: Eine Zeichenfolge, die einen Test innerhalb eines Testframeworks eindeutig identifiziert. Beispiele in vers. TestFrameworks bringen!

3.1.3 test execution

Zum Anderen muss dem Tool ein zweites Script zur Verfügung gestellt werden, welches eine Liste von zuvor in 3.1.2 beschriebenen qualifiedNames entgegennimmt, nur diese Tests ausführt, und das Ergebnis (auch wieder JUnit XML) an das Tool zurückmeldet, welches dann diesen Input zusätzlich an die gitnotes anhängt.

Absehen von diesen Anforderungen muss dem Tool lediglich der Pfad zu einem Ordner, der alle relevanten JUnit-XML Dateien enthält, mitgegeben werden.

Zur Vereinfachung dieses Prozesses benutzt das Tool die Datei "config.json", in der die Argumente zum Aufrufen der beiden Skripten, sowie der Pfad zu dem Ordner mit den JUnit-XML Dateien hinterlegt werden.

-> Hier Diagram mit Interaktion zwischen Skripten/Tool einfügen

3.2 Anforderungen an ein Framework

- kann JUnit XML
- enthält essenzielle Informationen für einen Test (genau ausführen!)
- kann tests einzeln aufrufen

3.3 Einschränkungen

- Annahme: keine deliberate bad actors; keine Authentifizierung
- An gitnotes angehängte Dateien spiegeln aktuellen Stand vom Code wieder, keine Veränderung der Codebase im Nachhinein

3.4 Codestruktur

4 Related Work

4.1 Andere Tools

Abgrenzung zu z.B Bazel/Gradle?

5 Fazit

5.1 Zusammenfassung der Ergebnisse

5.2 Möglichkeiten zur Verbesserung

In seiner aktuellen Form kann das Tool nicht entscheiden, ob die XML-Ausgabe der Tests auch tatsächlich aus dem aktuellen Stand des Codes entstammt, oder ob im Nachhinein Änderungen an dem zu testenden Code oder den Tests selber vorgenommen wurden, was die Aussagekraft des Tools stark beeinträchtigt.

Ein anderes Problem ist, dass das Tool alle Tests, die nicht an den letzten Schwung gitnotes anhängt sind, als nicht abgelaufen betrachtet, auch wenn einige Testfälle in einer vorherigen Iteration ausgeführt sein könnten, und die Änderungen, die seither im Code gemacht wurden diese Tests überhaupt nicht betreffen.

Eine Möglichkeit, dieses Problem zu beheben, wäre die Integration eines neuen Tools, welches eine Seletive Regression Testing / Regression Test Selection implementiert, und so dem Tool rückmelden kann, welche Tests tatsächlich neu ausgeführt werden müssen, und bei welchen der vorherige Stand noch gültig ist.

-> Ekstazi <https://github.com/gliga/ekstazi>

Literaturverzeichnis

